252-0027

Einführung in die Programmierung

Übungen

Woche 4: Types, Schleifen, Methoden

Joran Bühler
Departement Informatik
ETH Zürich

Inhalt

- Recap: Types
- Recap: Schleifen
- Coding with Joran
- Git Merge conflicts
- Strings
- Kahoot
- Vorbesprechung

Types

Primitive Typen in Java

Acht primitive Typen («primitive types»): für Zahlen, Buchstaben und Wahrheitswerte. Beispiele:

Name	Beschreibung	Beispiele
`int`	ganze Zahlen	`-2147483648`, `-3`, `0`, `42`, `2147483647`
`long`	grosse ganze Zahlen	`42`, `-3`, `0`, `92233720368547758O7`
`double`	reelle Zahlen	`3.1`, `-0.25`, `9.4e3`
`char`	(einzelne) Buchstaben	`'a'`, `'X'`, `'?'`, `'\n'`
`boolean`	Wahrheitswerte	`true`, `false`

Wer es genau wissen möchte: Java Language Specs §4.2

Fragen: Wozu evaluieren diese Ausdrücke?

1. `(30 * 2 + 5) % 7 / 3` ergibt 0
2. `485 % (12 + 6 * 2) – 4 * 5 / 2` ergibt -5
3. `(8 + 15 * 3 % 7) / 2 + 9` ergibt 14
4. `10 * (6 + 5 % 4) – 20 / 4 + 8` ergibt 73
5. `100 % (50 / 2) + 75 % (12 * 2) - 2` ergibt 1
6. `(9 + 3) * 2 – 18 % (5 + 2 * 3)` ergibt 17
7. `(6 * 3 + 2) % 7 – (14 / 2 + 4)` ergibt -5

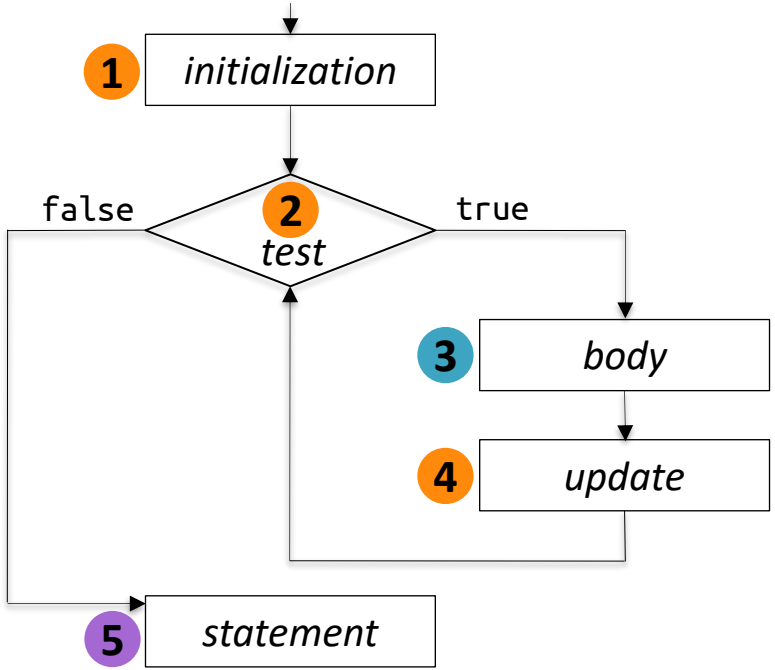
Schleifen

for-Schleife («for loop»): Kontrollfluss

```
for (initialization; test; update) {
    body
}
statement;
```

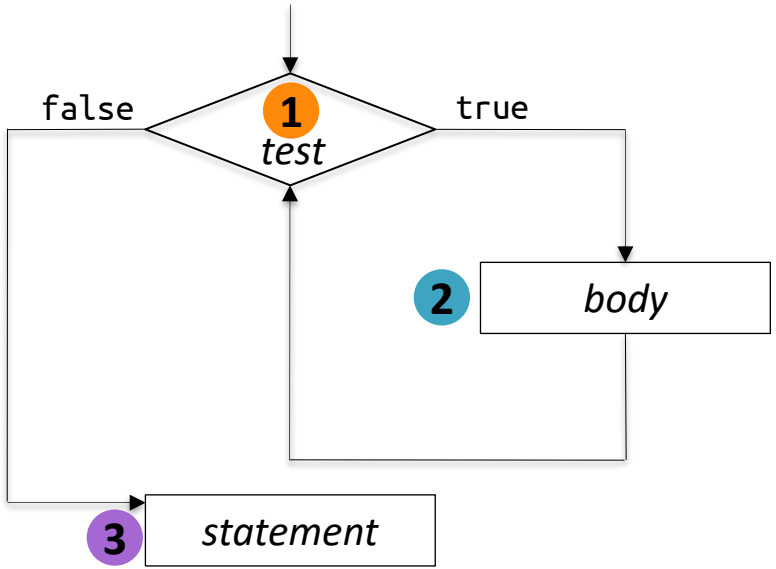
1 2 4 3 5

```
for (int i = 1; i <= 5; i = i + 1) {
 System.out.println(i); // 1 2 3 4 5
}
```



while-Schleife («while loop»)

```
while (test) {
    body
}
statement;
```



while-Schleife führt Rumpf so lange aus, wie *test* den Wert `true` ergibt.
In anderen Worten: die while-Schleife bricht ab, sobald *test* den Wert `false` ergibt.

Schleifen: While vs. For vs. Do-While

- **For-Loop:** Benutzen wir, wenn wir die Anzahl Iterationen bereits for der Ausführung des Loops kennen.
- **While:** Benutzen wir, wenn wir die Anzahl Iterationen nicht kennen und diese Abhängig von einer Bedingung ist.
- **Do-While:** Wird benutzt, wenn wir zuerst den Code (im Loop Body) ausführen und dann erst die Bedingung prüfen wollen.

Schleifen: For vs. While

- **While und Do-While können das gleiche.**
- **For und While aber auch!**

Schleifen: For vs. While – Beispiel 1

```java
public class MyClass {
    public static void main(String[] args){
        for(i = 0; i < 5; i++){
            System.out.println("i = " + i);
        }
    }
}
```

Schleifen: For vs. While – Beispiel 1

```java
public class MyClass {
    public static void main(String[] args){
        int i = 0;          // Initialisierung
        while(i < 5){       // Bedingung
            System.out.println("i = " + i);
            i++;            // Aktualisierung
        }
    }
}
```

Schleifen: For vs. While – Beispiel 2

```java
public class MyClass {
    public static void main(String[] args){
        int i = 1;
        while(fancyCheck(i)){
            int result = complexOperation(i);
            i = i * 42 - 15;
            System.out.println(result);
        }
    }
}
```

Schleifen: For vs. While – Beispiel 2

```java
public class MyClass {
    public static void main(String[] args){
        int i = 1;
        for(; fancyCheck(i); ){
            int result = complexOperation(i);
            i = i * 42 - 15;
            System.out.println(result);
        }
    }
}
```

Schleifen: Do-While?

- **Selten genutzt aber kann sehr nützlich sein!**

```java
public class MyClass {
    public static void main(String[] args){
        Scanner console = new Scanner(System.in);
        String pin;
        do {
            pin = console.nextLine();
        } while(!isPassword(pin));
        System.out.println("You logged in successfully!");
        console.close();
    }
}
```

Inkrement und Dekrement

Inkrement und Dekrement

- **Präfix: ++i**

```
int a = 5;
int b = ++a;  // Zuerst wird a inkrementiert, dann wird a zugewiesen.
// a = 6, b = 6
```

- **Postfix i++**

```
int a = 5;
int b = a++;  // Zuerst wird a zugewiesen, dann wird a inkrementiert.
// a = 6, b = 5
```

Diese Operatoren sind besonders nützlich in Schleifen, um beispielsweise einen Zähler zu erhöhen oder zu verringern.

Weitere Kurzformen

- **Prä-Inkrement («pre-increment»)** und **Prä-Dekrement («pre-decrement»)**: zuerst ändern, dann verwenden
  - `++variable` für `variable = variable + 1`
  - `--variable` für `variable = variable - 1`
- Veränderung mit beliebigen Wert (statt nur ± 1)
  - `variable += value` für `variable = variable + value;`
  - `variable -= value` für `variable = variable - value;`
  - `variable *= value` für `variable = variable * value;`
  - `variable /= value` für `variable = variable / value;`
  - `variable %= value` für `variable = variable % value;`

Achtung:

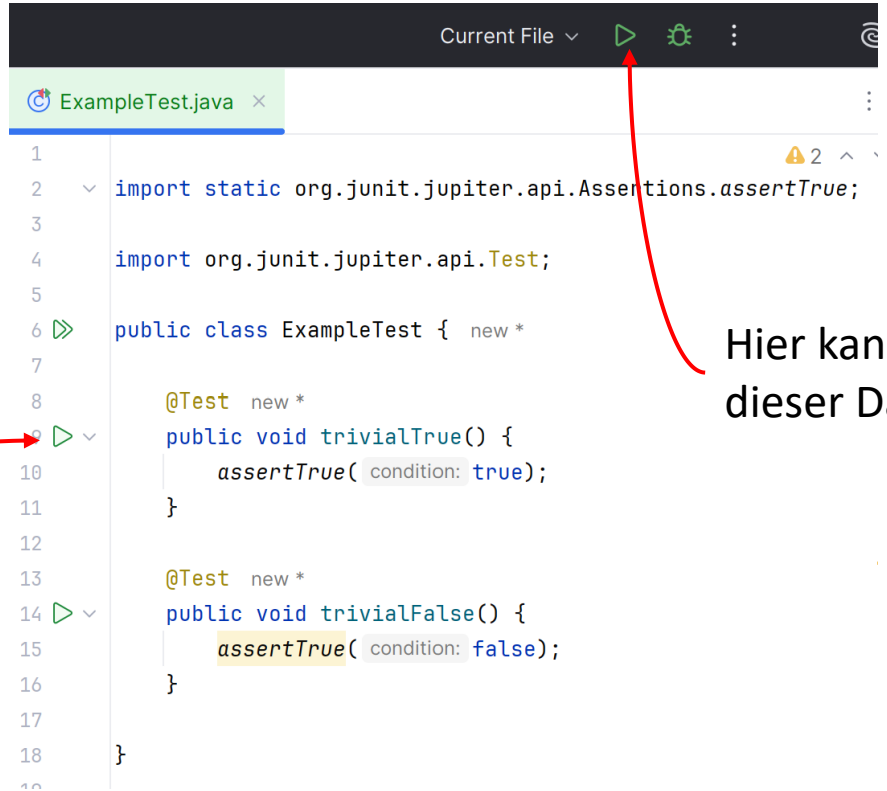
```
x += 3
x =+ 3
```

JUnit

JUnit Tests: Introduction

- **JUnit Test erlauben es Code zu testen.**
- **Jetzt: Tests benutzen**
- **(Später: Tests selbst schreiben.)**

JUnit Tests in IntelliJ



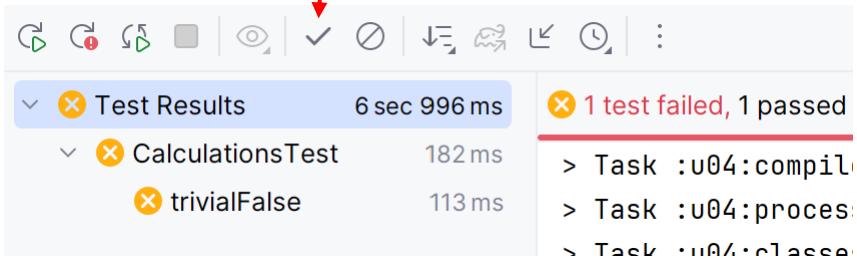
Hier kann man alle Tests dieser Datei laufen lassen

Hier kann man auch nur einen Test laufen lassen

JUnit Tests in IntelliJ

- **Standardmässig werden nur fehlgeschlagene Tests angezeigt:**

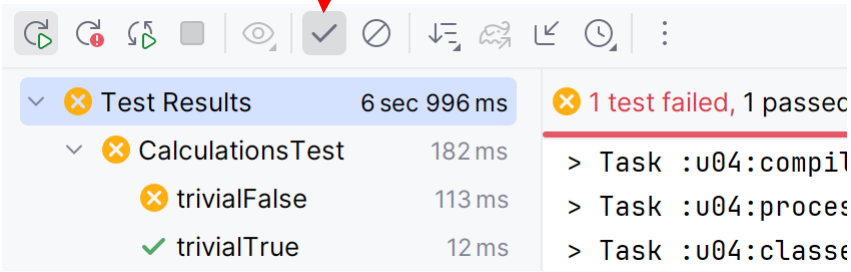
Hier kann man sich auch die bestandenen Tests anzeigen lassen



JUnit Tests in IntelliJ

- **Standardmässig werden nur fehlgeschlagene Tests angezeigt:**

Hier kann man sich auch die bestandenen Tests anzeigen lassen



Coding mit Joran

Methoden: Folgen und Reihen

Schreiben Sie ein Programm "Reihe.java", das eine Zahl N von der Konsole einliest und dann die folgende Summe berechnet:

$$\frac{1}{1^2} + \frac{1}{2^2} + \cdots + \frac{1}{N^2}$$

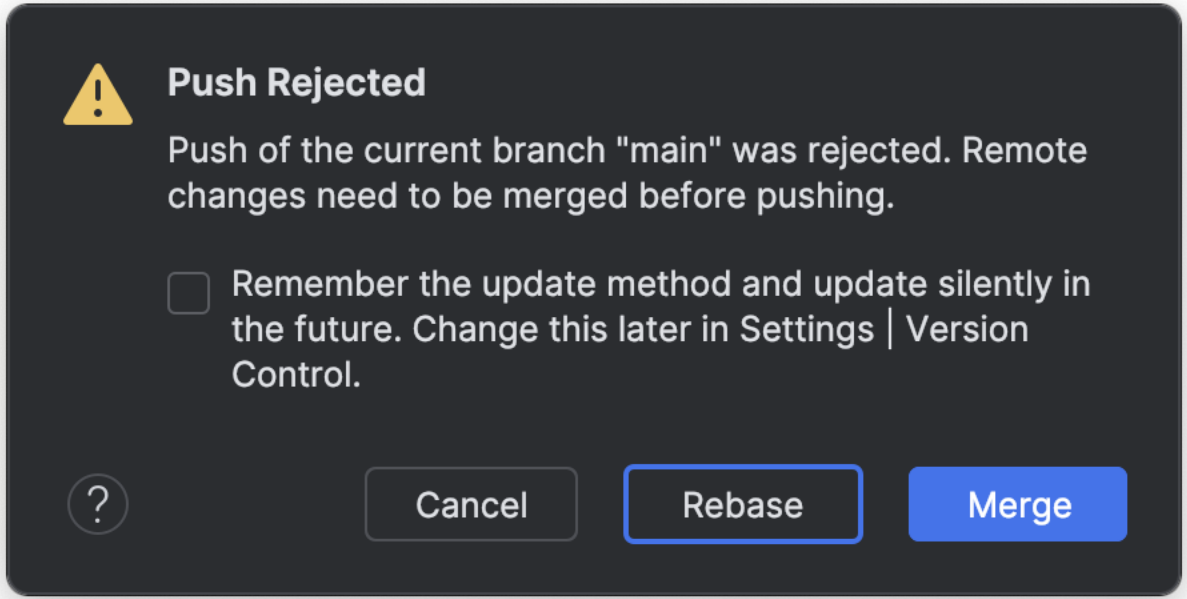
Beispiel: Für $N = 4$ sollte Ihr Programm ca. `1.42` ausgeben. Wie verhält sich diese Summe für grosse N?

Cool Tricks in IntelliJ

- **psvma: Generate Main function with String[] args**
- **sout: System.out.println()**
- **fori: For loop**
- **Ctrl + Alt + L / Cmd + Alt + L: Cleans up formatting**
- **Ctrl + Shift + Enter / Cmd + Shift + Enter: Finish Line**
- **(Usually) Ctrl + Shift + /: Comment/Uncomment Line**
  - Doesn’t work on some Laptops with swiss keyboard. -> set shortcut

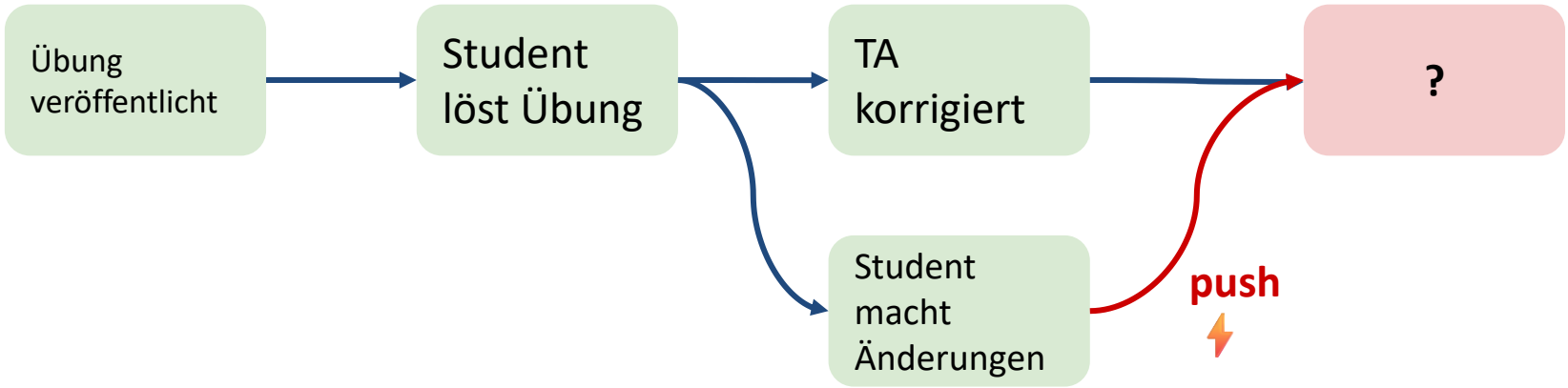
Git Merge Conflicts

Git sagt “Push Rejected”



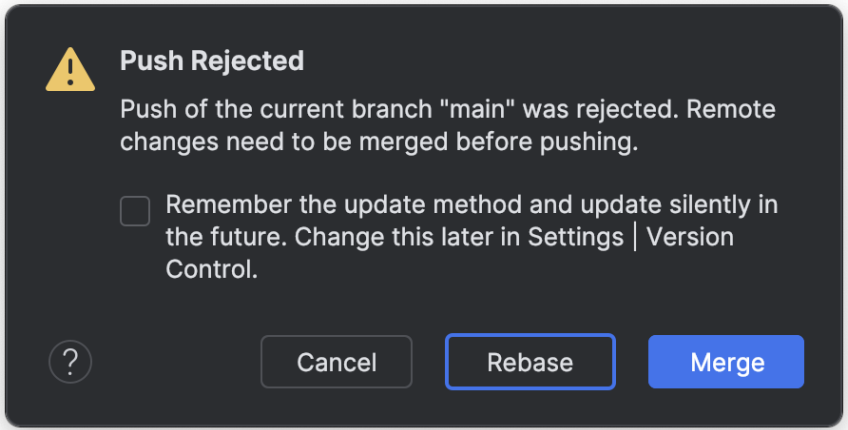
Git - Merge Conflicts

Student löst Übung gleichzeitig während TA korrigiert
→ Beim Versuch zu pushen, Konflikt!



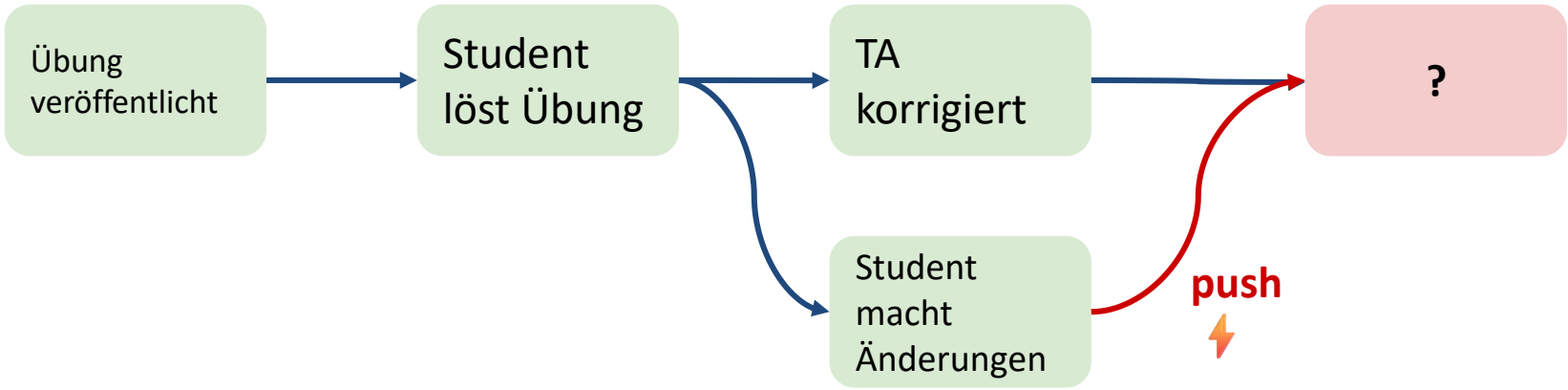
Zwei Optionen

- **Merge:** Kombiniert die zwei unterschiedlichen Versionen der Datei in eine kombinierte Datei.
- **Rebase:** Übernimmt die Änderungen auf dem Server und versucht deine Änderungen anzuwenden.

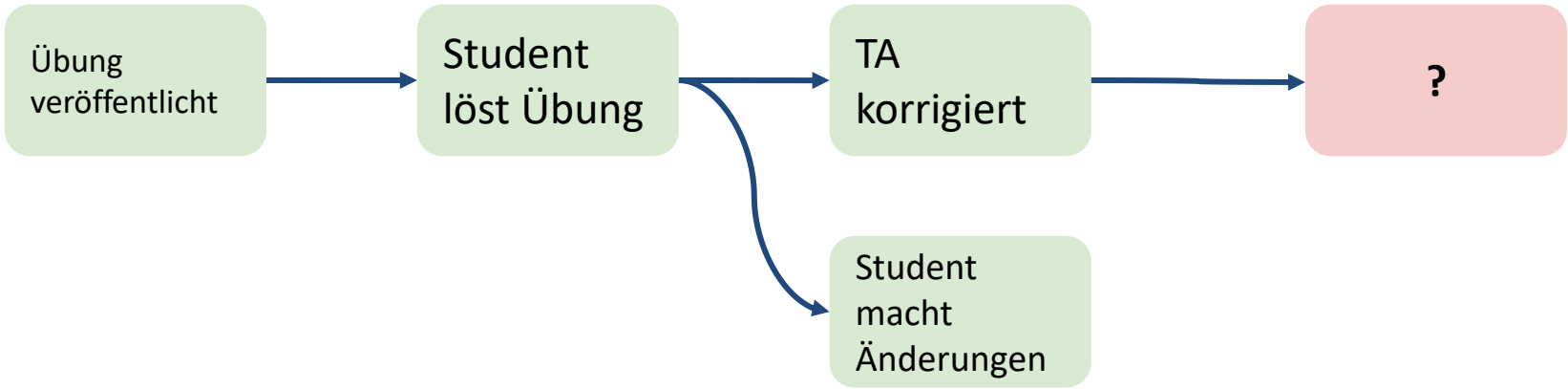


Git Merge

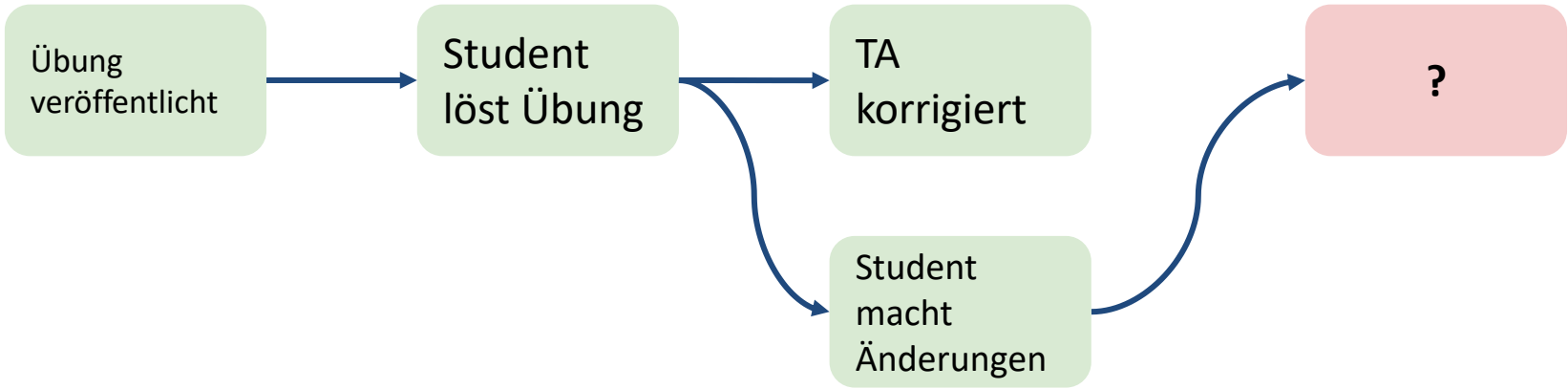
Git – Merge


Übung veröffentlicht
Student löst Übung
TA korrigiert
?
Student macht Änderungen
push

Git – Merge (Accept Theirs)

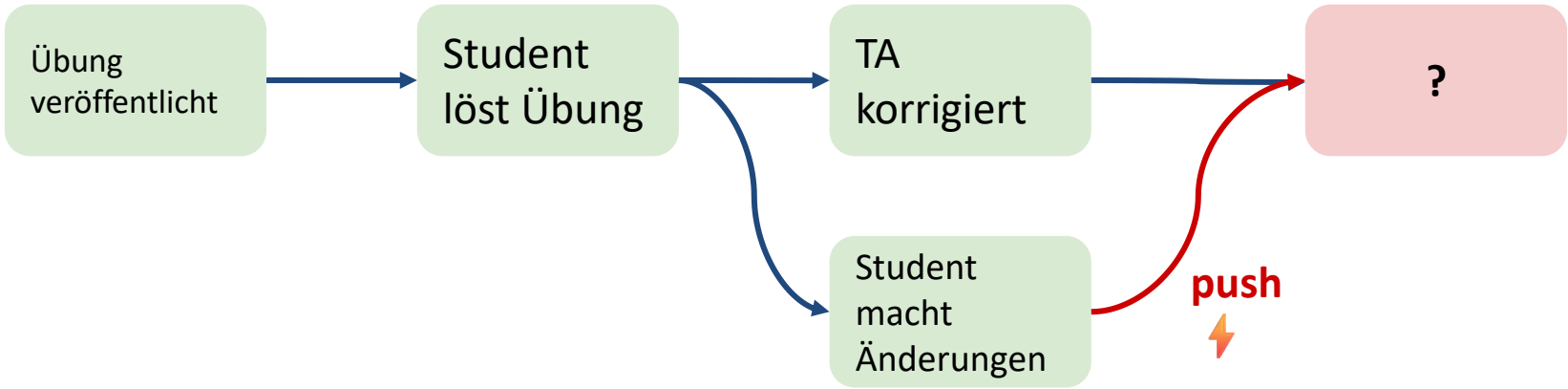


Git – Merge (Accept Yours)

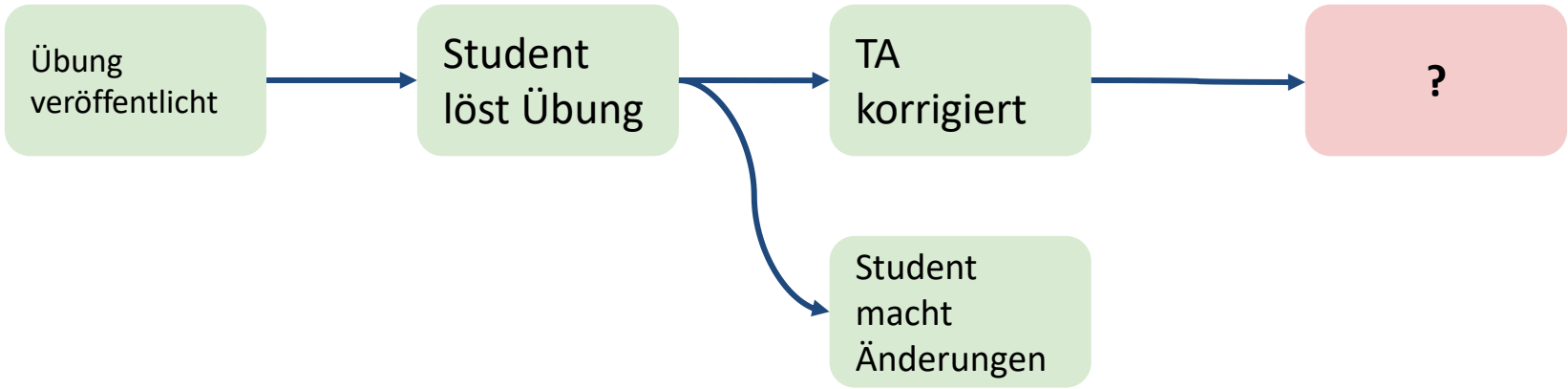


Git Rebase

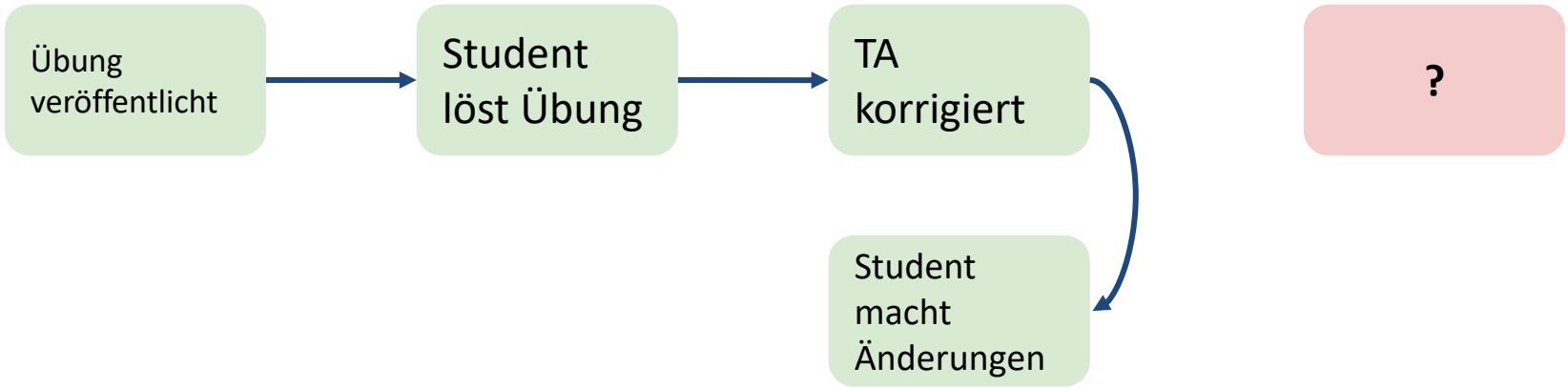
Git – Rebase


Übung veröffentlicht
Student löst Übung
TA korrigiert
?
Student macht Änderungen
push

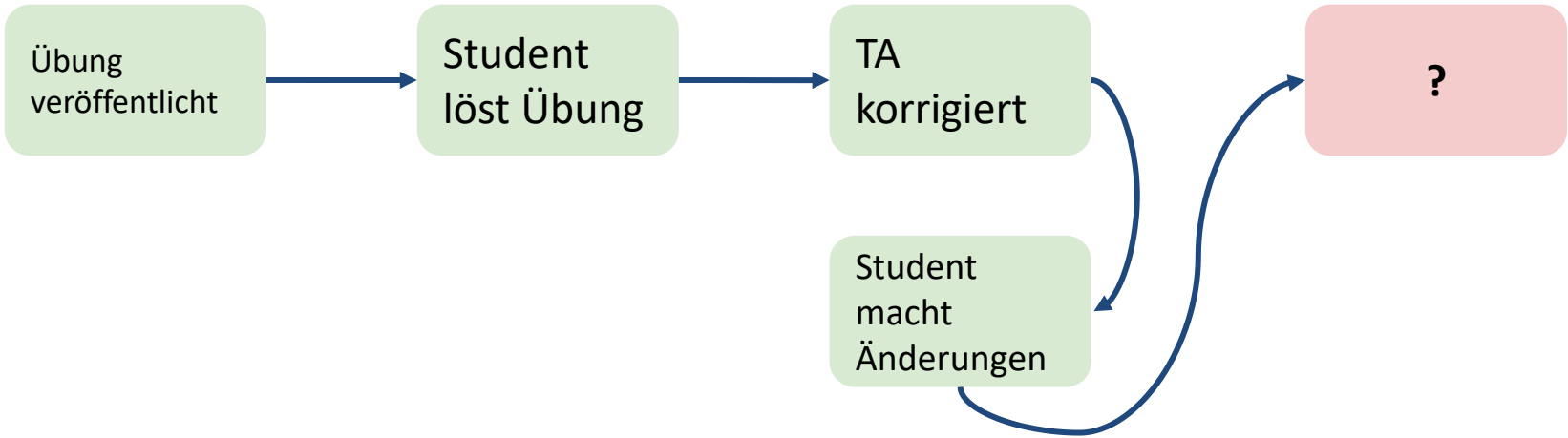
Git – Rebase


Übung veröffentlicht
Student löst Übung
TA korrigiert
?
Student macht Änderungen

Git – Rebase


Übung veröffentlicht
Student löst Übung
TA korrigiert
Student macht Änderungen
?

Git – Rebase

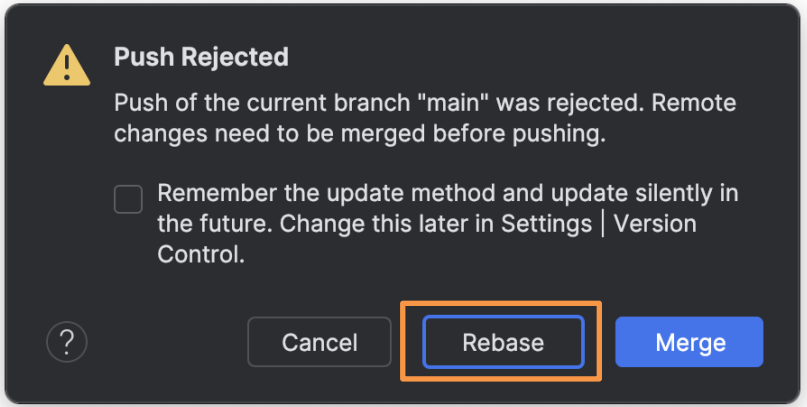

Übung veröffentlicht
Student löst Übung
TA korrigiert
?
Student macht Änderungen

Zwei Situationen

- **TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet.** 🥳
- **TA und Student haben dieselbe(n) Datei(en) bearbeitet.** 😓

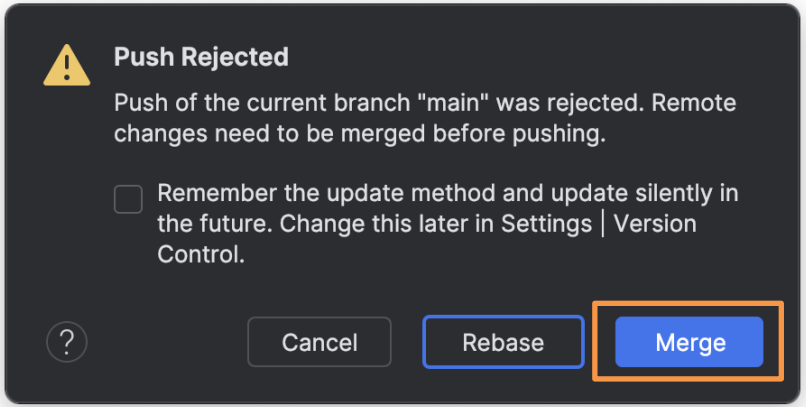
Zwei Situationen

- **TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet.** 🥳
  - ***Rebase*** wählen.
  - Änderungen sind gepushed.



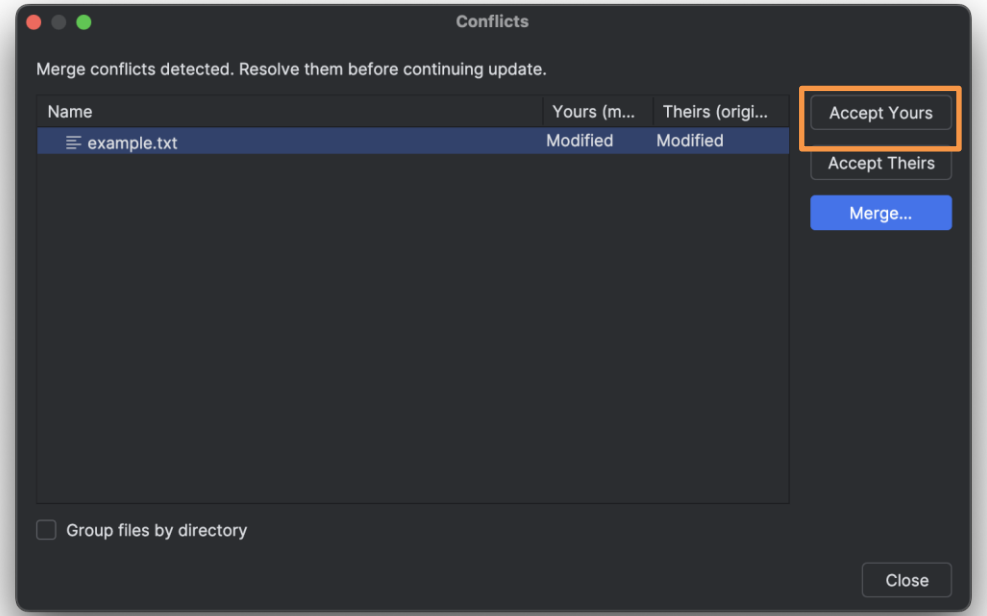
Zwei Situationen

- **TA und Student haben dieselbe(n) Datei(en) bearbeitet.** 😓
  - ***Merge*** wählen.



Zwei Situationen

- **TA und Student haben dieselbe(n) Datei(en) bearbeitet.** 😓
  - ***Merge*** wählen.
  - ***Accept Yours*** wählen.
  - Änderungen gepushed.



Strings

Strings: Funktionalität

- Strings sind Objekte: Objekt `s` vom Typ `String`
  - Strings sind keine Arrays! Arrays folgen bald!
- Standardbibliothek enthält viel Funktionalität für Strings
  - Immer vorhanden, ohne `import java.util.*;`
  - Methoden werden mit dot-Notation aufgerufen: `s.method(parameters)`
    (führe Methode `method` für `s` aus, wende Methode `method` auf `s` an)
  - Ergebnis (Rückgabewert der Methode) kann sein: `String`, `int`, `boolean`, `char`
- Strings sind **unveränderbar** («**immutable**»)
  - Zuweisung verändert nicht den String: Variable verweist auf neuen String
  - Alle Methoden lassen String unverändert

String-Methoden, die String liefern

Name der Methode	Beschreibung der Methode

`substring(i1, i2)`
or
`substring(i1)`
	Ein neuer String:
	Der Substring von `i1` (**inklusive**) bis `i2` (**exklusive**);
	falls kein `i2` übergeben wird: bis Ende des Strings

`toLowerCase()`	Ein neuer String mit nur Kleinbuchstaben
`toUpperCase()`	Ein neuer String mit nur Grossbuchstaben
`stripLeading()`	Ein neuer String ohne Leerzeichen am Anfang
`stripTrailing()`	Ein neuer String ohne Leerzeichen am Ende

Substrings

```
String name = "Bob Dylan";
String firstName = name.substring(0, 3); // "Bob"
String lastName = name.substring(4); // "Dylan"
String firstCharacter = name.substring(0, 1); // "B"
name = name.toLowerCase(); // "bob dylan"
```

Indizes 0, 1, 2

Von Index 4 bis Ende

name:	B	o	b		D	y	l	a	n
Index	0	1	2	3	4	5	6	7	8

Achtung: `name.substring(0, 1)` ist nicht das gleiche wie `name.charAt(0)`

- `name.substring(0, 1)` hat Wert `"B"` und ist vom Typ `String`
- `name.charAt(0)` hat Wert `'B'` und ist vom Typ `char`

String-Methoden, die `int` liefern

Name der Methode	Beschreibung der Methode
`length()`	Länge des Strings (Anzahl Zeichen)

`indexOf(s, fromIndex)`
`indexOf(c, fromIndex)`
`indexOf(s)`
`indexOf(c)`
	Index wo String `s` oder Zeichen `c` zum ersten Mal nach Index `fromIndex` im String auftaucht (-1 falls nicht gefunden);
	falls `fromIndex` nicht übergeben wird, von Anfang des Strings

```
String s = "S. Beckett";
int indexBeck = s.indexOf("Beck");          // 3
int eFirstOccurrence = s.indexOf('e');      // 4
int eSecondOccurrence = s.indexOf('e', 5); // 7
```

String-Methoden, die boolean liefern

Name der Methode	**Beschreibung der Methode** bei Aufruf `s1.method(s2)`
`equals(s2)`	ob `s1` und `s2` die gleichen Buchstaben enthalten
`equalsIgnoreCase(s2)`	ob `s1` und `s2` die gleichen Buchstaben enthalten, ohne Berücksichtigung von Gross- und Kleinschreibung
`startsWith(s2)`	ob `s1` mit den Buchstaben von `s2` anfängt
`endsWith(s2)`	ob `s1` mit den Buchstaben von `s2` endet
`contains(s2)`	ob `s1` irgendwo `s2` enthält

Hinweis zu == und `equals`

- `==` nur für Basistypen (z.B. `int` oder `char`), nicht für Strings
- `==` für Strings wird später erklärt!

Wo findet man diese Informationen?

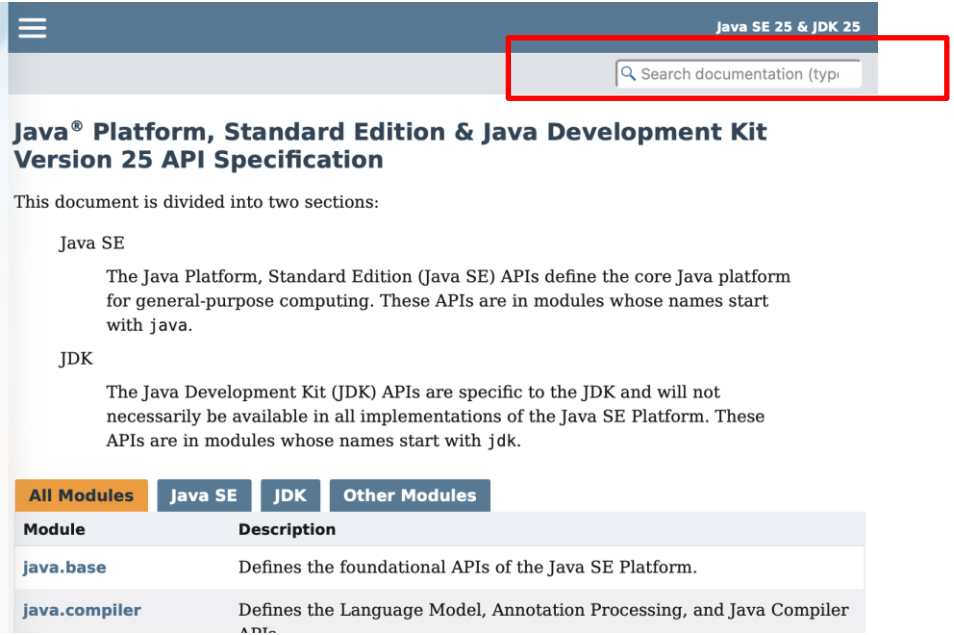
- **In der Java Documentation 25**



https://docs.oracle.com/en/java/javase/25/docs/api/

Wo findet man diese Informationen?

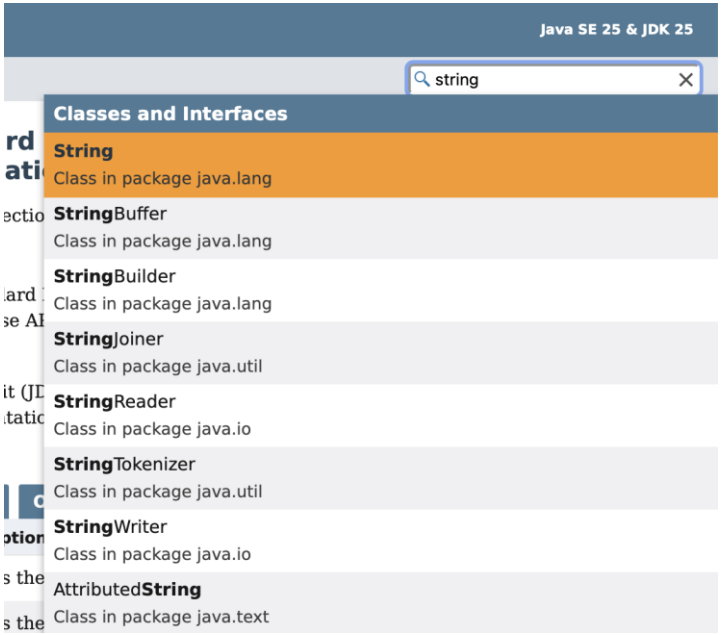
- In der Java Documentation 25



https://docs.oracle.com/en/java/javase/25/docs/api/

Wo findet man diese Informationen?

- **In der Java Documentation 25**

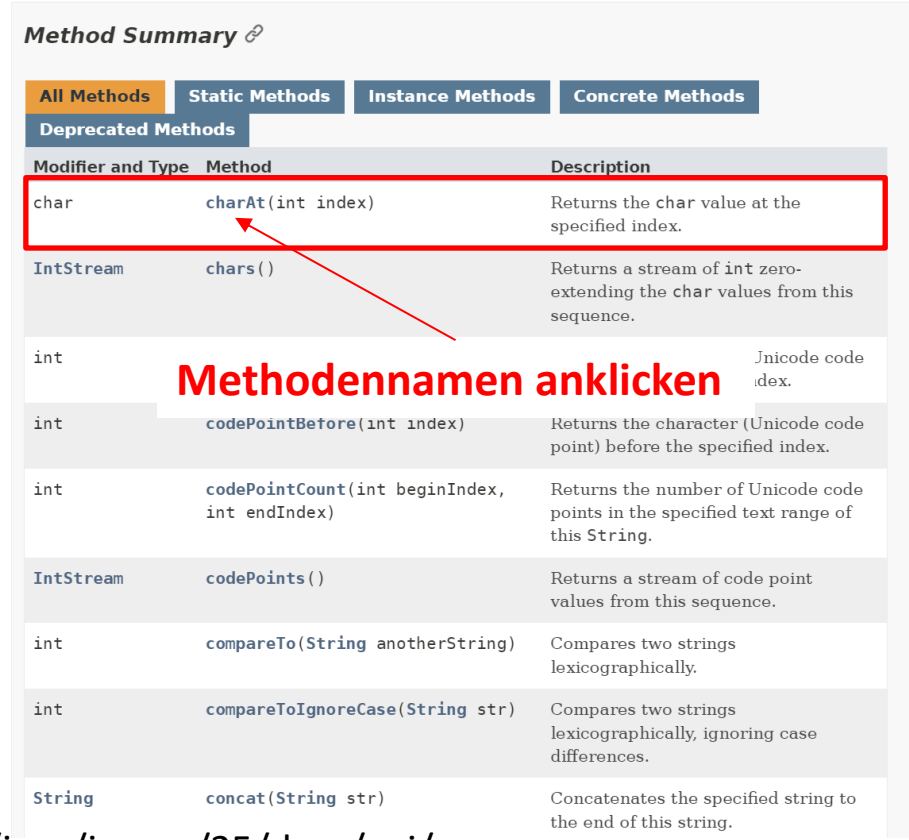


https://docs.oracle.com/en/java/javase/25/docs/api/

Wo findet man diese Informationen?

- **In der Java Documentation 25**





https://docs.oracle.com/en/java/javase/25/docs/api/

Wo findet man diese Informationen?

- **In der Java Documentation 25**

charAt

```
public char charAt(int index)
```

Returns the `char` value at the specified index. An index ranges from `0` to `length() - 1`. The first `char` value of the sequence is at index `0`, the next at index `1`, and so on, as for array indexing.

If the `char` value specified by the index is a surrogate, the surrogate value is returned.

Specified by:

`charAt` in interface `CharSequence`

Parameters:

`index` - the index of the `char` value.

Returns:

the `char` value at the specified index of this string. The first `char` value is at index `0`.

Throws:

`IndexOutOfBoundsException` - if the `index` argument is negative or not less than the length of this string.

https://docs.oracle.com/en/java/javase/25/docs/api/

String Beispiele

```java
public class MyClass{
    public static void main(String[] args){
        String str1 = "Java21";
        String str2 = "EPROG2024";
        String result = str1.substring(0, 2) + str2.substring(str2.length() - 4);
    }
}
```

Output: Ja2024

Name der Methode	Beschreibung der Methode
`substring(i1, i2)` or `substring(i1)`	Ein neuer String: Der Substring von `i1` (**inklusive**) bis `i2` (**exklusive**); falls kein `i2` übergeben wird: bis Ende des Strings

String Beispiele

```java
public class MyClass{
    public static void main(String[] args){
        String str = "Substring";
        int result = str.indexOf('S') + str.lastIndexOf('g') - str.length();
    }
}
```

Output: -1

String Beispiele

```java
public class MyClass{
    public static void main(String[] args){
        String str = "Programmierung";
        String result = str.charAt(0) + str.substring(2, 5).toLowerCase() + str.charAt(str.length() - 1);
    }
}
```

Output: Pogrg

Kahoot

https://create.kahoot.it/share/u4-loops-methoden-short-circuit-strings/a4cb9c10-92a0-4ead-8ac7-385114beb364

Vorbesprechung

Aufgabe 1: Binärdarstellung



Schreiben Sie ein Programm "Binaer.java", das eine positive Zahl Z einliest und dann die Binärdarstellung druckt.

Hinweis: Finden Sie zuerst die grösste Zahl k, so dass die Zweierpotenz $K = 2^k$ kleiner oder gleich Z ist. Dies ist das erste 1-Bit, danach können Sie die Zweierpotenz abziehen und absteigend durch die Zweierpotenzen gehen. Falls $K = 2^{k-1}$ grösser als die verbleibende Zahl ist, so ist das nächste Bit 1, sonst 0, usw.

Beispiel: Für $Z = 14$ sollte Ihr Programm `1110` ausgeben.
Beachten Sie, dass wir hier kein Zweierkomplement verwenden und nur positive Zahlen repräsentiert werden sollen.

Aufgabe 2: Grösster gemeinsamer Teiler



Schreiben Sie ein Programm "GGT.java", das den grössten gemeinsamen Teiler (ggT) zweier ganzer Zahlen mithilfe des Euklidischen Algorithmus berechnet. Hierbei handelt es sich um eine iterative Berechnung, die auf folgender Beobachtung basiert:

1. Wenn x grösser oder gleich y ist und sich x durch y teilen lässt, dann ist der ggT von x und y gleich y;
2. andernfalls ist der ggT von x und y der gleiche wie der ggT von y und $x \,\%\, y$.

Üben Sie diesen Algorithmus zuerst von Hand an ein paar Beispielen und schreiben Sie dann das Java Programm.

Beispiel: Für $x = 36$ und $y = 44$:

$$ggT(\underset{x}{36}, \underset{y}{44}) \overset{2.}{=} ggT(44, \underbrace{36 \,\%\, 44}_{36}) \overset{2.}{=} ggT(36, \underbrace{44 \,\%\, 36}_{8}) \overset{2.}{=} ggT(8, \underbrace{36 \,\%\, 8}_{4}) \overset{1.}{=} 4$$

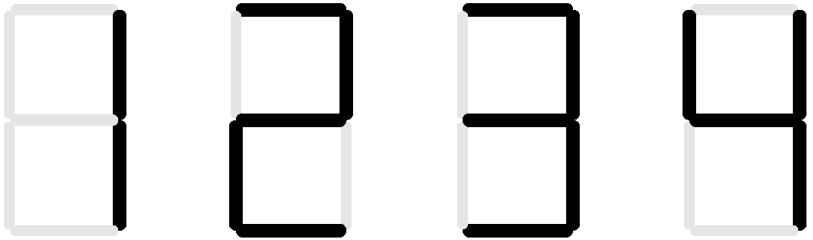
Aufgabe 3: Zahlenerkennung

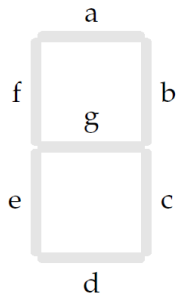


Für diese Aufgabe verwenden wir einen String um die erleuchtenden Segmente einer Sieben-segmentanzeige zu kodieren. Die Segmente sind, wie im Bild gezeigt, von a bis g nummeriert. Die Kodierung einer möglichen Anzeige ist ein String, in welchem der Buchstabe 'x' genau dann vorkommt, wenn das 'x'te Segment der Anzeige erleuchtet ist. Zum Beispiel wird die Zahl 2 kodiert durch 'abged'. Zur Einfachheit darf angenommen werden, dass kein Buchstabe mehr als einmal in der Kodierung vorkommt und dass nur die Zahlen 0 bis 9 kodiert werden.

Schreiben Sie ein Programm "Zahlen.java", das einen String, der eine Anzeige kodiert, einliest und die kodierte Zahl als Integer ausgibt. Überlegen Sie wie viele IF Blöcke benötigt werden um jede Zahl zu erkennen.

Tipp: Sie können `str.contains("a")` verwenden, um zu überprüfen, ob ein String `str` den Buchstaben 'a' enthält.





Aufgabe 4: Berechnungen



2. Implementieren sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele

   - `magic7(2,5)` gibt `true` zurück.
   - `magic7(7,9)` gibt `true` zurück.
   - `magic7(5,6)` gibt `false` zurück.

   Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

Aufgabe 4: Berechnungen



3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe an einem Vielfachen von `12` ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal `2` grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Method `false` zurückgeben.

   - `fast12(12)` gibt `true` zurück.
   - `fast12(14)` gibt `true` zurück.
   - `fast12(10)` gibt `true` zurück.
   - `fast12(15)` gibt `false` zurück.

Aufgabe 5: Scrabble



In dieser Aufgabe sollen Sie Scrabble-Steine legen, mittels ASCII-Art auf der Konsole. Vervollständigen Sie die Methode `drawNameSquare` in der Klasse `Scrabble`. Diese Methode nimmt einen Namen als String-Parameter und soll den Namen als in einem Quadrat angeordnete Scrabble-Steine auf der Konsole (`System.out`) ausgeben. Wenn z.B. der String `Alfred` übergeben wird, sollte folgendes Bild ausgegeben werden:

```
+---+---+---+---+---+---+
| A | L | F | R | E | D |
+---+---+---+---+---+---+
| L |                   | E |
+---+                   +---+
| F |                   | R |
+---+                   +---+
| R |                   | F |
+---+                   +---+
| E |                   | L |
+---+---+---+---+---+---+---+
| D | E | R | F | L | A |
+---+---+---+---+---+---+
```

Aufgabe 5: Scrabble

Ihr Code braucht nur Namen der Länge 3 oder länger unterstützen. Für einen Namen mit Länge 3, z.B. `Jim`, sollte die Ausgabe so aussehen:

```
+---+---+---+
| J | I | M |
+---+---+---+
| I |   | I |
+---+---+---+
| M | I | J |
+---+---+---+
```

Beachten Sie, dass Ihr Programm keinerlei andere Ausgabe als das Scrabble-Quadrat machen darf.

Aufgabe 6: Operatoren - Grundlagen



Aufgabe 6: Operatoren: Grundlagen

In dieser Aufgabe untersuchen Sie, in welcher Reihenfolge Java die Bestandteile eines Ausdrucks auswertet. Beachten Sie dabei insbesondere die Priorität und Assoziativität der Operatoren, Ganzzahldivision, String-Verkettung, Typumwandlungen und die Kurzschlussauswertung boolescher Ausdrücke. Die folgenden Codefragmente sind voneinander unabhängig und werden jeweils innerhalb einer `main`-Methode ausgeführt.

Bestimmen Sie für jede Teilaufgabe die exakte Konsolenausgabe, ohne den Code auszuführen. Berücksichtigen Sie dabei auch Leerzeichen, Zeilenumbrüche und den Datentyp des Ergebnisses. Kennzeichnen Sie jedes Leerzeichen in Ihrer Antwort explizit mit ␣. Beginnt die Konsolenausgabe eine neue Zeile, beginnen Sie auch in Ihrer Antwort eine neue Zeile. Sie dürfen Ihre Antworten anschliessend durch Ausführen des Codes kontrollieren.

a) `System.out.println(18 - 3 * 4 + 17 % 5);`

Ausgabe: _____

_____

Aufgabe 7: Operatoren - Vertiefung



Aufgabe 7: Operatoren: Vertiefung

In dieser Aufgabe kombinieren Sie mehrere Regeln zur Auswertung von Java-Ausdrücken. Achten Sie zusätzlich auf Klammerung, gemischte Ganzzahl- und Gleitkommaarithmetik, Präfix- und Postfix-Inkremente, den bedingten Operator, Zuweisungen, Casts sowie die Darstellung von `double`-Werten. Wie in Aufgabe 6 sind alle Codefragmente voneinander unabhängig und werden jeweils innerhalb einer `main`-Methode ausgeführt.

Bestimmen Sie wiederum die exakte Konsolenausgabe, ohne den Code auszuführen. Kennzeichnen Sie jedes Leerzeichen mit ␣ und übernehmen Sie jeden Zeilenumbruch der Konsolenausgabe in Ihre Antwort.

a)
```
System.out.println(48 / (4 / 2) + 7 % 4);
```

Ausgabe: _____

_____

Schleifen

Was gibt diese Methode aus?

```
public static void main (String[] args) {
    int f = 0; int g = 1;

    for (int i = 0; i < 15; i++ ) {
        System.out.print(" " + f);
        f = f + g;
        g = f - g;
    }
    System.out.println();
}
```

i	f	g
	0	1
0	0	1
0	**0**	1
0	1	1
0	1	0
1	**1**	0
…	…	…

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Prüfungsaufgabe FS22 – 3/40 Punkte

Gegeben sei eine Methode `main` in einer Java Klasse.

```
public static void main(String[] args) {
    /* body */
}
```

Die folgenden Anweisungen sollen als "Body" (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden. Geben Sie für jede Anweisung an, was für eine Ausgabe erzeugt wird – entweder was gedruckt wird, oder ob ein Laufzeitfehler auftritt (schreiben Sie "Exception"), oder ob der Compiler einen Fehler feststellt (schreiben Sie "Compile-Fehler"). Achten Sie auf die korrekte Formatierung der verschiedenen Typen, also z.B. 7.0 statt 7 für eine reelle Zahl (`double`).

1. `System.out.println(11 + 16 / 4 * 2 +  (4 + ">") + 4 * 2);`

   194>8

---

2. `System.out.println(20 / 10 % 6 + 3 / 2 + (double) 5 / 4 + 3 / 4);`

   4.25

---

3. `System.out.println( ( 11 % 4 ) >  2  && 9 > (16 / (2 / 4)) && 1 % 8 <  0);`

   Exception: Division by 0
